

MDAAI: A Protocol for Governing AI-Assisted Development

Eris Margeta Kurdali

TEJL — Tvornica Elektroničkih Jedinica i Logike, Croatia
eris.margeta@gmail.com • TEJL.com • TEJL.hr

Abstract

When an AI-assisted coding task stops between an edit and its verification, the next participant needs more than the patch. They need to know what was authorized, what remains unresolved, and which observations support the recorded result. MDAAI governs AI-assisted development through rules for authority, scope, evidence, history, and review. This paper uses interruption as a lens on that broader purpose, following a hypothetical date-parser repair through two template representations: a work-order document with a synchronized index, and a task registry with conditional supporting records. The comparison distinguishes internal record consistency from reconciliation with the current implementation, and prescribed narrative records from judgment about when a separate explanation is needed. The contribution is a documentary design account, not a measurement of improved task performance. Testing must establish whether recording and reconciliation prevent enough recovery and acceptance errors to justify their cost.

1. The task after the interruption

A maintainer asks an AI coding agent to repair a date parser. The agent adds a regression test and edits the implementation, then the session ends. The next participant can see the patch. That does not tell them whether the new test ran, whether existing behavior changed, or whether the maintainer authorized a broader API revision. A note saying “parser fixed” compresses several unanswered questions into one apparent result.

MDAAI is a protocol for governing AI-assisted development. It organizes authority, scope, evidence, history, and review across project work. This paper focuses on how its records connect an authorized assignment, ongoing work, and observations supporting acceptance, so another participant can inspect them together. A template supplies the project files and update rules through which people and agents apply the protocol. Community members can extend the open-source templates or develop their own; the two examined here illustrate different representations of the method. [1–3]

The interrupted repair provides a path through authorization, verification, and possible closure. It also exposes differences between a formal work-order document and a task registry. The account draws on fixed revisions of the public protocol documentation and both templates, with source locations given at the end. The shared workflow is an explanatory synthesis of these materials; specific recording obligations belong to the selected template.

2. Connect the assignment to its evidence

The starting point is a human instruction with a bounded objective. In both studied templates, a direct explicit request can authorize the work without a second approval of the same assignment. The participant records where the request came from, what change it permits, and how completion will be judged. This gives the later reader something against which to compare the implementation. A stored plan or task description does not supply additional permission merely by mentioning more work. [2, 3]

The work record then develops with the task. It identifies the responsible participant, describes current progress, and points to the relevant observations. Acceptance criteria describe the behavior sought; evidence describes what was actually checked. A planned test belongs with the next action until someone runs it. An observed result needs enough context to identify the case, the code or artifact examined, and the outcome. The templates encode these distinctions in their work-order and task-record instructions. [2, 3]

These connections make the record more useful than a status label alone. A reviewer can follow a completion claim back to a criterion, its supporting observation, and the authorized assignment. A failure at any link changes the interpretation: a passing test of an earlier implementation cannot establish the result of an untested edit; a test that checks only rejection cannot establish preserved valid-input behavior.

Resumption adds a further step: compare the recorded account with the project as it now exists. The task-registry template explicitly directs participants to read the relevant unfinished task and linked evidence, inspect the current working tree or runtime, and reconcile stale or contradictory information before treating recovery as complete. A useful next action narrows that inspection, but does not replace it. Section 4 distinguishes this recovery rule from synchronization between records. [3]

3. An inspectable repair

Consider this hypothetical request: “Correct the parser’s leap-day handling so it rejects 2025-02-29 and still accepts 2024-02-29. Keep the public API and existing valid-input behavior unchanged. Make and test the change locally.” Under the Gregorian calendar, the first date is invalid and the second is valid. Assume the existing API already defines an invalid-input response. The request permits a bounded repair, not a redesign of the calling interface.

Call the illustrative task *PARSER-A*, assigned to the coding agent under the maintainer’s request. Its scope reference is that request, and its work record identifies the parser and relevant tests. Suppose the agent has observed the new regression fail because the invalid date was accepted, written a candidate repair, and stopped before post-edit verification. Table 1 is an illustrative projection, not a template schema; the failure and interruption are stipulated scenario conditions, not results collected for this paper.

The task remains unfinished. Its next action is concrete: inspect the current parser and test changes, confirm that they still implement the authorized repair, then run the invalid-date regression and the valid-input and affected caller checks. That sequence matters if another participant has edited the parser since the interruption. The stipulated failure describes the earlier code, not the candidate now on disk. Under the task-registry template, this comparison is part of the documented recovery step. [3]

The two criteria prevent an easy but incomplete repair from appearing sufficient. Rejecting every input would satisfy the single invalid-date case while violating C2. The record therefore connects each proposed acceptance claim to a different observation.

Closure is conditional on the checks actually supporting both criteria and the applicable project requirements; the listed checks pro-

Table 1 Acceptance record at the stipulated interruption. Bracketed entries are evidence still to be obtained, not links to executed tests.

Acceptance criterion	Status at interruption	Observation needed before acceptance
C1. 2025-02-29 is rejected through the parser's existing invalid-input response.	Pre-edit regression failure stipulated; post-edit check pending.	Post-edit regression result: [not yet available]. Retain the selected case, command, source identity, exit status, and observed response.
C2. 2024-02-29 retains its expected value and return type; the public call signature and established valid-input fixtures remain unchanged.	Unchecked after the edit.	Valid-input and affected caller checks: [not yet available]. Retain the expected/observed values and interface review alongside the results.

vide acceptance evidence for preservation, not exhaustive proof of unchanged behavior. The participant would attach the resulting observations, identify the checked revision or artifact, and report the supported result. If the signature had changed, even successful date checks would not meet the unchanged-API requirement. The work would remain unresolved until repaired or explicitly rescoped. For an authorized scope change, the work-order template requires specification and acceptance updates before changed execution; the task-registry template prevents a task entry from expanding the request's authority. [2, 3]

If a defect were discovered after closure, a new linked task, PARSER-B, would identify the earlier repair and the newly found problem. The earlier observations would remain historical evidence, while the new record would make the unresolved defect visible. Both studied templates preserve closed results through linked follow-up work, although their correction records differ. The distinction lets a reader see what supported the earlier decision without mistaking it for a guarantee that the product remains correct. [2, 3]

4. Two ways to represent the method

A project applies MDAAI by selecting and configuring a template for its authority, scope, records, and verification commands. The studied templates organize task records around either a work order or a task registry. [1–3]

The work-order template uses a saved assignment document. Before implementation, the participant creates and registers a Work Order containing authorization, the specification, and planned verification, then opens it for execution. During work, the document accumulates progress and observations; its registry entry supplies the corresponding index and status. Both must be kept synchronized. Closure requires actual verification and a review of the project planning document, called Project Elaboration. For PARSER-A, the request supplies authorization, C1 and C2 enter the specification and acceptance criteria, and pending checks enter the verification plan. The progress record carries the next action; executed results later support closure. [2]

The task-registry template gives Project Elaboration a narrower role: scope and sequence. A single file, `TASKS.json`, holds task state, acceptance criteria, evidence references, next actions, and transition history. It is the designated source for those facts, so the planning document must not maintain a competing status list. PARSER-A's entry would link the request as its scope reference, record C1 and C2, put the pending checks in its next action, and link observations as they become available. The task entry and evidence are normally sufficient; an additional durable note is used when a decision, defect cause, or handoff would otherwise be hard to understand. [3]

The work-order arrangement prescribes a dedicated document and coordinated document/index updates. Those updates are required, not transactional. The registry arrangement avoids that duplication and leaves participants to judge when a separate explanation is necessary. It still requires structured task fields and transition history. This comparison concerns separate narrative records and

their maintenance, not total governance strictness or an observed efficiency ranking. [2, 3]

Record consistency and recovery also address different questions. Synchronizing a work order and its index keeps two records in agreement; the registry template's recovery rule requires comparison with the current implementation. The work-order template also addresses project-state desynchronization through an operator-initiated comparison with the last known stable state. [2, 3]

For later correction, the work-order template creates a linked Corrective Work Order and appends correction links and current correction status to the original registry entry without replacing its closed outcome. The task-registry template uses a new linked task and reflects the defect in current acceptance reporting. [2, 3]

5. What the design establishes—and what remains to test

Persistent instructions and externalized context are important overlapping ideas, not features unique to MDAAI. Calboreanu's Context Engineering methodology uses staged collaboration, ranked context roles, a versioned operator-authority document, and a master reference for continuity across sessions. MDAAI shares the concern with making working context explicit. This paper's narrower focus is the connection between an authorized repository task, its evolving acceptance record, and reconciliation with the current project. [2–4]

Recording more is not automatically better. In their study of repository context files, Gloaguen and colleagues found that generated files generally reduced task-resolution performance, while developer-written files offered modest gains in most tested settings; both introduced additional cost. Their experiment did not test MDAAI, but it makes the cost of reading and maintaining instructions a necessary part of evaluating this design. A useful handoff must preserve what the next participant needs without burying it in repetitive records. [5]

The present evidence is documentary and illustrative. It establishes what the inspected materials prescribe and allows readers to examine the design, but does not measure adherence, productivity, or defect reduction. The author designed MDAAI. In practice, the proposed continuity depends on participants maintaining the records and judging the relevance of evidence; writable files alone do not authenticate their contents. These limits make independent task execution and outcome assessment necessary before drawing effectiveness conclusions.

A focused evaluation could interrupt matched coding tasks after an edit but before verification, then compare resumption under ordinary issue records and under a configured MDAAI template, using the same model, tools, and permissions. Independent checks would assess code correctness and whether completion claims had current supporting observations. Recovery errors would include continuing outside the assigned scope, overlooking an unchecked criterion, or relying on evidence from a different revision. Total time, model usage, and human effort spent creating and maintaining records would be measured alongside those errors. This would test the complete

workflow; isolating the value of its record structure would require a further comparison controlling instruction content.

6. Conclusion

MDAAI connects authority, work records, acceptance evidence, and subsequent correction within a development-governance protocol. Examining an interrupted task shows how those connections can support inspection through synchronized work-order records or a task registry with additional context when needed. Whether they improve recovery and completion claims enough to justify their maintenance remains an empirical question.

Source note

The documentary corpus is protocol release 1.0.0 [1], the MDAAI 1.0 template family, release 1.0.1 [2], and the MDAAI 2.0 template family, release 2.0.1 [3]. Protocol and template releases are independently versioned. Full immutable Git URLs are retained in `references-v2.bib`; the two external papers remain at their inspected arXiv v1 revisions.

Protocol scope and template adoption follow `README.md` and `website/content.json`, “Apply the protocol with a template.” Within each template’s `PROJECT-INTERNAL/GOVERNANCE/`, the work-order sources are `AI-INSTRUCTIONS.md`, Article 5.1(4) (scope amendment), 5.1(5)–(8) (verification, closure, and registry updates), and Article 7 (state synchronization); and `WORK-ORDER-PROTOCOL.md`, “Required fields by phase” and “Registry operation and terminal correction.” The task-registry sources are `AUTHORITY.md`, “Scope of authorization,” and `RECORDS.md`, “One state source,” “Optional durable records,” and “Recovery and supersession.” Template claims are bounded by these instructions.

AI assistance disclosure

AI tools assisted drafting and documentary source inspection and produced the editorial and scientific critiques. The author retains responsibility for the manuscript.

References

- [1] Eris M.K.: MDAAI public protocol documentation. Protocol release 1.0.0; inspected 5 October 2026 (n.d.). <https://github.com/Eris-Margeta/mdaai/tree/1265670beae34680053dcd5cc9416b25e4593913>
- [2] Eris M.K.: MDAAI first-generation template family. Release 1.0.1; inspected 5 October 2026 (n.d.). <https://github.com/Eris-Margeta/mdaai-template-1/tree/3263e95a30dee5c1699568f9f5bf8e493208d3b1>
- [3] Eris M.K.: Portable MDAAI 2.0 template. Release 2.0.1; inspected 5 October 2026 (n.d.). <https://github.com/Eris-Margeta/mdaai-template-2/tree/3ae05fb46d92caf93d4ba63f6ddc6b95c04d6ea0>
- [4] Calboreanu, E.: Context Engineering: A Practitioner Methodology for Structured Human-AI Collaboration. arXiv preprint, version 1 (2026). <https://doi.org/10.48550/arXiv.2604.04258> . <https://arxiv.org/abs/2604.04258v1>
- [5] Gloaguen, T., Mündler, N., Müller, M., Raychev, V., Vechev, M.: Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? arXiv preprint, version 1 (2026). <https://doi.org/10.48550/arXiv.2602.11988> . <https://arxiv.org/abs/2602.11988v1>